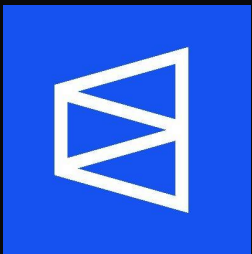




Security Assessment Final Report



Polymarket CCIP + PRs

July 2026

Prepared for Polymarket

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	4
Assessment Methodology.....	5
Findings Summary.....	6
Severity Matrix.....	6
Detailed Findings.....	7
Low Severity Issues.....	8
L-01: Optimistic Oracle cannot report `Other` in atomic NegRisk markets.....	8
L-02: Bridged combinatorial positions are minted without their legs.....	10
Informational Severity Issues.....	12
I-01: Outdated documentation after recent changes.....	12
Disclaimer.....	15
About Certora.....	15

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Polymarket CCIP + PRs	https://github.com/Polymarket/polymarket-v2	Initial: 264fcb4 Fix review: c4af693	EVM

Project Overview

This document describes the security review of **Polymarket CCIP + PRs**. The work was undertaken from **June 23rd, 2026** to **July 3rd, 2026**.

The following contract list is included in our scope:

```
src/bridge/CcipBridge.sol
src/libraries/Ids.sol
src/modules/abstract/BaseModule.sol
src/modules/abstract/ModuleErrors.sol
src/modules/abstract/OracleModule.sol
src/modules/migration/BinaryMigrationMixin.sol
src/modules/migration/NegRiskMigrationMixin.sol
src/modules/BinaryModule.sol
src/modules/CombinatorialModule.sol
src/modules/NegRiskModule.sol
src/oracle/abstract/OracleAggregatorErrors.sol
src/oracle/OracleAggregator.sol
src/routers/Router.sol
```

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Polymarket's next-generation prediction market smart contract system. Manages position tokens (ERC1155), collateral wrapping/unwrapping, market resolution via modular oracles, order matching, and cross-chain bridging. Built with Foundry and Solady.

Positions are represented as structured IDs that encode module routing, condition identity, and outcome index directly in the token ID — eliminating storage lookups and enabling pure derivation of all relationships.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	–	–	–
Medium	–	–	–
Low	2	2	0
Informational	1	1	1
Total	3	3	1

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
L-01	Optimistic Oracle cannot report `Other` in atomic NegRisk markets	Low	Acknowledged
L-02	Bridged combinatorial positions are minted without their legs	Low	Acknowledged
I-01	Outdated documentation after recent changes	Informational	Fixed

Low Severity Issues

L-01: Optimistic Oracle cannot report `Other` in atomic NegRisk markets

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/oracle/modules/OptimisticOracleModule.sol	Status: Acknowledged	

Description:

Atomic NegRisk requests are initialized with a result length equal to the number of real outcomes, while the synthetic **Other** condition is derived separately by **NegRiskModule** after all real outcomes are resolved.

OracleAggregator can represent an **Other** outcome as an all-zero result vector over the real outcomes. After those real outcomes are finalized as NO, **NegRiskModule** derives the synthetic **Other** condition as YES. However, **OptimisticOracleModule** treats any request with more than one result entry as a NUMERICAL UMA request and only accepts integer prices in the range of the real outcome indices. The settled price is then converted into a one-hot payout vector by **OptimisticOraclePayoutLib**, so every valid OO price selects one real outcome and none can produce the all-zero vector needed for **Other**.

If an OO-backed atomic NegRisk market resolves to **Other**, the OO path cannot submit the correct result. A mistaken real-outcome proposal could finalize if undisputed, while a natural extra index for **Other** would be rejected as an invalid price and leave the request unresolved until another resolution path is used.

The practical severity is Low because this issue is somewhat out of scope for this audit, which was not focused on **OptimisticOracleModule**, and because that module is expected to be replaced.

Recommendations:

Do not configure **OptimisticOracleModule** as the reporter for atomic NegRisk markets where synthetic **Other** can be the final outcome. Route those resolutions through the planned managed

reporter path, such as **EOARporter**, or add explicit OO support for an **Other** price that maps to an all-zero real-outcome vector before enabling OO for these markets.

Customer Response:

Acknowledged. A synthetic **Other** resolution will be provided from something more managed like an **EOARporter** module.

L-O2: Bridged combinatorial positions are minted without their legs

Severity: Low	Impact: Low	Likelihood: Medium
Files: src/bridge/abstract/BridgeBase.sol	Status: Acknowledged	

Description:

When positions are bridged, the bridge message only carries each position id and amount. The destination side decodes that payload and calls `mintFromBridge` on the destination module:

```
// Mint positions directly - no condition preparation needed
BaseModule(moduleAddr).mintFromBridge(recipient, pos.positionId, pos.amount);
```

This is enough for binary and NegRisk positions because their ids encode the data needed by the module. It is not enough for combinatorial positions. `CombinatorialModule` stores the canonical leg array in `legs[conditionId]`, and most combinatorial operations reconstruct their behavior from that local mapping.

`CombinatorialModule.prepareCondition` stores the leg array, but `mintFromBridge` does not require or populate it. It only validates that the bridged position belongs to the combinatorial module and has a YES or NO outcome, then mints the token. As a result, a destination chain can receive a valid combinatorial ERC1155 balance whose condition id has no corresponding legs entry in storage.

Any later operation that needs the legs will fail or be unusable until the exact leg array is prepared on the destination chain. This includes `redeem`, `compress`, `splitOnCondition`, `mergeOnCondition`, `splitOnEvent`, `mergeOnEvent`, `convertOnEvent`, `extract`, `inject`, `convertToYesBasket`, `mergeFromYesBasket`, and `unwrap`.

The impact is that users who bridge combinatorial positions can be left with positions that are valid balances but not operational on the destination chain. The funds are not lost because anyone can call `prepareCondition` with the correct legs, but normal users may not know the original leg array or how to reconstruct it from the source chain. Until the mapping is filled

correctly, users are effectively blocked from redeeming, compressing, refining, converting, or unwrapping those positions.

Recommendations:

Include the combinatorial leg data in the bridge flow when bridging combinatorial positions, and have the destination chain prepare the condition before or during `mintFromBridge`.

Because carrying full leg arrays in every bridge message can be more expensive, an acceptable alternative is to run a protocol-maintained off-chain bot that watches bridged combinatorial position ids, fetches or reconstructs their source-chain legs, and promptly calls `prepareCondition` on the destination chain before users need to interact with those positions.

Customer Response:

Acknowledged.

Informational Severity Issues

I-01: Outdated documentation after recent changes

Files:

README.md, docs/position-ids.md, docs/modules.md, docs/auth.md, docs/migration.md, docs/bridge.md, docs/oracle.md, docs/router.md, docs/architecture.md, src/oracle/README.md, src/routers/Router.sol, src/modules/CombinatorialModule.sol

Description:

Recent changes updated the ID format, resolver authorization, bridge replay behavior, migration resolution flow, and Router support for `CombinatorialModule` operations. The code now reflects those changes, but several comments and documentation files still describe the previous behavior.

Affected documentation and comments:

1. Position ID layout and module IDs

Affected locations:

README.md:10, CLAUDE.md:235, docs/position-ids.md:8, docs/position-ids.md:24, docs/position-ids.md:69, docs/position-ids.md:97, docs/position-ids.md:126

Issue:

These sections still describe the old ID layout with an 80 bit reserved field and no resolution chain field. Current code in `src/libraries/Ids.sol` includes a resolution chain field and reduces the reserved field size. The position ID documentation also omits the `CombinatorialModule` module ID, even though `src/libraries/ModuleIds.sol` defines it and Router now exposes combinatorial operations.

Recommendation:

Refresh the ID layout diagrams, field descriptions, API tables, and derivation examples so they match the current encoding. Also document the `CombinatorialModule` module ID and clarify that generated module IDs include the configured resolution chain.

2. Resolver, replay, and migration result semantics

Affected locations:

`docs/modules.md:19`, `docs/modules.md:87`, `docs/modules.md:119`, `docs/auth.md:35`,
`docs/auth.md:121`, `docs/migration.md:25`, `docs/migration.md:33`, `docs/migration.md:52`,
`docs/migration.md:60`, `docs/bridge.md:89`

Issue:

The docs still refer to `onlyOracle` style behavior and imply broad idempotency for result reporting. Current `BinaryModule` and `NegRiskModule` behavior is more specific: resolver callers are checked against the encoded resolution chain, matching resolver replays revert, matching bridge replays return silently, and conflicting replays revert. Migration result reporting also has special bridge behavior because bridge supplied payouts are verified against legacy CTF results.

Recommendation:

Update the module, auth, migration, and bridge docs to describe the current resolver role model, the bridge role exception, replay behavior, and migration verification flow.

3. Router and `CombinatorialModule` surface

Affected locations:

`docs/router.md:9`, `docs/router.md:37`, `README.md:76`, `docs/architecture.md:5`,
`docs/architecture.md:17`, `docs/architecture.md:71`, `docs/architecture.md:97`,
`CLAUDE.md:13`, `CLAUDE.md:153`, `src/routers/Router.sol:89`,
`src/modules/CombinatorialModule.sol:245`

Issue:

Router docs and source comments still describe the Router as covering base module operations and neg-risk horizontal operations only. Current Router code also exposes combinatorial wrappers and `combinatorialCollateralReturn`, which uses the registered `CombinatorialModule` target. Architecture docs also still describe combinatorial support as future or incomplete in places, while it is now a first class module surface.

Recommendation:

Update the Router and architecture docs to include the combinatorial wrapper surface, the collateral return flow, and the fixed registered module target. Update the affected source comments so NatSpec matches the current external API.

4. Bridge transport and payload documentation

Affected locations:

`docs/bridge.md:3`, `docs/bridge.md:5`, `docs/bridge.md:20`, `docs/bridge.md:27`,
`docs/bridge.md:61`, `docs/bridge.md:96`, `docs/bridge.md:116`, `docs/auth.md:80`,
`docs/auth.md:130`, `docs/architecture.md:63`, `docs/architecture.md:67`, `CLAUDE.md:13`,
`CLAUDE.md:153`, `CLAUDE.md:493`, `CLAUDE.md:634`

Issue:

These files document LayerZero and LzBridge as currently supported in the tree, but the current source tree only contains `CcipBridge`. The bridge payload docs also mention condition and event message types that are no longer present in `CrossChainTypes.sol` or `BridgePayloads.sol`.

Recommendation:

Revise the docs to describe the currently supported CCIP bridge only. Also update the payload and message type sections to match the current bridge library surface.

Recommendations:

Perform a focused documentation sync pass for the files above. The updates should be broad and behavior oriented: align diagrams, role descriptions, replay rules, migration notes, router coverage, architecture diagrams, and bridge message descriptions with the current source.

Customer Response:

Fixed in [c4af693](#).

Fix Review:

Fix confirmed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.